

Build CampusLink from scratch

A practical manual for planning, creating, testing and deploying a campus placement platform.

WHAT YOU WILL BUILD	HOW THIS GUIDE HELPS
Student, recruiter, campus and administrator workspaces	Features, permissions and the order to implement them
A complete placement lifecycle	Profiles, readiness, matching, scheduling, offers and analytics
A local prototype and a production deployment	Technology choices, folders, commands, configuration and release checks

Scope and assumptions

This guide interprets the request as a manual for creating the **CampusLink project from the beginning**. It uses the project in C:/CampusLink as the reference. The implementation snapshot was reviewed on **8 October 2026**.

Current describes the inspected repository. **Build step** explains how to recreate the behavior. **Recommended** identifies a proposed improvement or process, not a feature already present.

The core application can run locally without paid cloud services. Production uses PostgreSQL, private file storage and a real email provider. External AI and historical outcome prediction are optional extensions.

Start here

First complete one reliable journey: a campus registers, a student adds a profile, a recruiter publishes an approved drive, the student applies, the team schedules a round, and an offer is accepted and tracked. Expand preparation and analytics after that journey works.

No application code or existing credentials were changed to create this manual.

The build order and reading map

STAGE	BUILD FIRST	PAGES
Plan	Define roles, rules and the minimum release	3
Choose the stack	Understand each tool and system boundary	4-6
Create the foundation	Folders, local setup and database records	7-9
Protect access	Authentication, campus scope and approvals	10
Build evidence	Profiles, private documents and assessments	11-12
Run placement	Matching, drives, scheduling and interview rounds	13-14
Complete the lifecycle	Notifications, offers and joining	15
Show results	Analytics, frontend integration and optional AI	16-17
Verify and release	Tests, hosting, maintenance and handover	18-21

Dependency sequence

Requirements -> Types + database -> Authentication
 -> Student profiles + verified evidence
 -> Drives + eligibility -> Applications
 -> Scheduling + rounds -> Offers + joining
 -> Notifications + analytics -> Deployment

Create the shared navigation and page shell early, but connect each screen to a working backend operation as you go. A polished dashboard cannot replace a correct permission check or transaction.

Minimum release versus later work

Minimum release: role access, registered campuses, student profiles, recruiter approval, approved drives, applications, scheduling, offer progress, private uploads and basic analytics.

After the minimum release: richer preparation content, Google sign-in, communication coaching, voice support, model evaluation and scaling to more campuses.

Completion check

You can explain the core user journey, its records and the next unfinished stage without depending on a cloud AI service.

Step 1 - Define the project requirements

Problem: placement teams need one reliable place to manage student readiness, eligible applicants, recruiter visits, interview rounds and offer outcomes.

ROLE	MAIN ACTIONS	ACCESS BOUNDARY
Student	Maintain profile; practice; upload evidence; apply; respond to offers	Own records and available approved opportunities
Recruiter	Maintain organization; propose drives; review matches; run rounds; release offers	Owned drives and authorized applicants
Campus team	Register institution; approve recruiter access; support students; schedule; verify joining	Students and operations in its own campus
Administrator	Manage account access, campuses, question content and audits	Server-issued administrator permission

Write these rules before building screens

- A campus must exist before its students can join. Campus and recruiter accounts need approval. Email verification is not required by the current application.
- A role is established on the server. Students cannot grant themselves verified skills, points, staff access or administrator rights.
- A drive must pass review and scheduling before appearing as an active student opportunity. Hard eligibility gates are separate from ranking.
- An offer follows a valid selection outcome. Campus document verification is required before recording joining.

Documents to create

Create a requirements list, a role/permission matrix, screen sketches, an entity list and acceptance scenarios. Give each requirement an ID, a responsible role, a success condition and its failure cases.

Example acceptance scenario: given a recruiter-owned confirmed drive and an eligible student in an allowed campus, the student can apply once; an unrelated recruiter cannot view the application.

Completion check

Every important action has an authorized actor, required input, stored result and clear rejection condition.

Step 2 - Technologies for the frontend

TECHNOLOGY	USED FOR	WHY / WHERE
React	Reusable interface components	Forms, cards, tables, dialogs and role dashboards in src/components and src/features.
Next.js	Website framework and page routing	App Router under src/app; layout, build output and same-origin API rewrites. See official reference [1].
TypeScript	Checked contracts and safer refactoring	Used in both frontend and backend. Define roles, profiles, drives and API payloads.
Tailwind CSS + CSS	Styling and responsive layout	The repository includes Tailwind 4 and feature-specific CSS. Keep reusable sizes, colors and spacing consistent.
React Hook Form + Zod	Form state and input validation	Handle errors, required fields and structured schemas. Repeat validation on the server.
Axios	Browser-to-API requests	Central client handles base path, cookies, CSRF token and errors.
TanStack Query	Fetches data, caching and refresh	Load dashboards; invalidate affected queries after updates. Avoid duplicating server data in a global store.
Zustand	Small shared client state	Session display and shared interface state; server identity remains authoritative.
Recharts	Analytics charts	Readiness, pipeline progress, offer conversions and campus outcomes.
Lucide / Framer Motion	Icons and motion	Consistent controls and optional animation. Preserve readable labels and reduced-motion behavior.

Repository compatibility

The inspected manifest declares Node.js 24.x, Next.js ^15.5.0, React ^19.1.0, TypeScript ^5.7.0 and Tailwind ^4.0.0. These are project declarations, not a claim that they are the latest releases. Use the existing lockfile to reproduce the project.

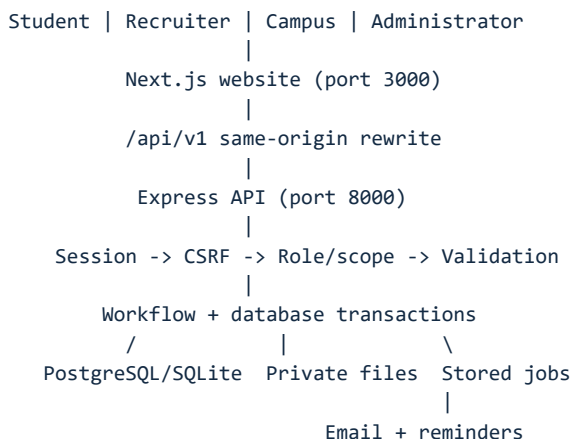
For a genuinely new project, choose compatible supported releases and test the combination. Current Next.js scaffolding can differ from this repository; review its generated files rather than assuming the same routing or dependency versions.

Technologies for the backend and services

TECHNOLOGY	USED FOR	WHY / WHERE
Node.js 24.x	JavaScript/TypeScript server runtime	Matches the project engine and supports its node:sqlite development database.
Express 5	HTTP API and request middleware	Authentication, documents, service operations, drives and analytics under /api/v1. See [2].
Zod	Server payload validation	Reject malformed values and fields clients are not permitted to set.
PostgreSQL + pg	Production persistence and transactions	Accounts, workflow records and scope queries. Current code uses the pg driver, not Prisma or MongoDB.
SQLite	Local database without a hosted account	Persistent development fallback when DATABASE_URL is empty; node:sqlite is used directly.
Node crypto	Passwords, random tokens and hashes	Salted script passwords; random session tokens; hashed session IDs. No JWT dependency is required here.
Helmet / rate limits / cookies	HTTP headers and request protection	Add to origin validation, session checks, CSRF checks and authorization; they do not replace those controls.
Multer + pdf-parse	Upload handling and PDF text extraction	Bound file size and validate signatures; store files privately and extracted metadata in records.
Supabase Storage	Private production documents	Backend holds credentials and authorizes file access. Private buckets require controlled downloads. See [3].
Resend + email templates	Actual email delivery	Queued messages and password recovery. Local outbox is used during development.
tsx / esbuild	Development and backend builds	Run TypeScript scripts and bundle server entry points into CommonJS output.
Vercel / Render	Frontend / persistent API hosting	Repository deployment configuration uses these services. Render Express deployment is documented in [4].

Recommended later: browser/API test tooling, a shared rate-limit store and a durable queue for multiple worker replicas. They are not currently included as an automated test suite or distributed worker system.

Step 3 - Understand the architecture



Example: student submits an application

- The browser sends a service request through its frontend /api/v1 URL with the session cookie and write CSRF token.
- Express resolves the server account, validates the operation and checks the drive state, student campus and eligibility.
- The workflow checks duplicates, creates the application and related notification/email job inside a database transaction.
- The response updates the interface. Query invalidation reloads relevant lists; the background worker handles email delivery separately.

Where the responsibilities belong

Frontend: user input, display, accessible navigation and helpful validation. **Backend:** trusted identity, permissions, scoring, workflow transitions and consistent writes. **Database:** durable records and transactional updates. **Storage:** private file bytes. **Workers:** retryable email, reminders and cleanup.

Current boundaries to preserve

The Express server is the authoritative API. Next.js forwards requests; do not copy business rules into a second backend without a reason. Keep server secrets away from NEXT_PUBLIC_* variables. Browser permissions and hidden buttons are never sufficient authorization.

Completion check

You can trace each user action from a screen to the API, the saved records and any background side effects.

Step 4 - Create the folder structure

```
CampusLink/
src/
  app/           # Routes, layout, page styles
  features/      # Student/recruiter/campus/admin screens
  components/    # Shared controls and layouts
  services/api/  # Remote client and request handling
  services/      # Typed domain/service interfaces
  hooks/         # Data and interaction hooks
  store/         # Session/interface state
  types/         # Shared contracts
  utils/         # Scoring and reusable rules
  config/        # Public configuration
  mocks/         # Existing sample/content adapters
server/
  app.ts         # Express routes and middleware
  index.ts       # Startup and workers
  auth.ts        # Passwords and sessions
  db.ts          # Database adapter and migrations
  workspace.ts   # Placement operations
  recruitment.ts # Applications, rounds and offers
  services.ts    # Operation dispatch and policies
  storage.ts     # Private file provider
  mail.ts        # Email queue and delivery
  reminders.ts   # Periodic reminders
  nlp.ts / ml.ts # Text relevance / optional model
scripts/        # Dev startup, CLI demos
public/         # Public images, fonts and logos
output/         # Generated reports
.env.example
server/.env.example
package.json / next.config.mjs
render.yaml / vercel.json
```

Create modules in layers

Start with types and configuration, then database and authentication. Add a domain operation, its API allowlist, the client function, a query/mutation hook and finally the screen. Reuse small presentation components between roles while keeping each role dashboard independent.

The current app includes a catch-all route in `src/app/[...slug]/page.tsx`. A new build can instead use explicit role route folders, but that is an alternative structure rather than a description of the existing repository.

Completion check

A new feature has an obvious place for its contract, trusted operation, API client and screen.

Step 5 - Initialize and run locally

A. Starting a genuinely empty project

Install Node.js 24.x, Git and an editor. Work in a new empty folder so scaffolding does not overwrite CampusLink. The following bootstrap uses current scaffolding; it does not recreate the exact existing dependency versions [1,2].

```
npx create-next-app@latest campuslink-new
# Select TypeScript, ESLint, Tailwind, src/ and App Router.
cd campuslink-new
npm install express pg zod axios cookie-parser cors dotenv
npm install helmet express-rate-limit multer pdf-parse
npm install @tanstack/react-query zustand react-hook-form
npm install @hookform/resolvers recharts lucide-react
npm install resend @react-email/components
npm install -D tsx esbuild @types/express @types/pg
npm install -D @types/cookie-parser @types/cors
npm install -D @types/multer @types/pdf-parse
```

Create server/app.ts with Express and a GET /api/v1/health route. Create server/index.ts to load configuration and listen on port 8000. Set up a Next.js rewrite from /api/v1/:path* to the backend. Add backend development, build and start scripts before using those script names. Optional animation packages can wait.

B. Running the existing CampusLink repository

```
cd C:\CampusLink
npm install
# Copy only if each destination does not already exist:
Copy-Item .env.example .env
Copy-Item server/.env.example server/.env
npm run dev
```

Open **http://localhost:3000**. The startup script starts or reuses a healthy API on port 8000. For separate terminals, use **npm run server:dev** and **npm run dev:frontend**. Preserve existing .env values; do not replace credentials with sample values.

Local configuration

Frontend: NEXT_PUBLIC_APP_ENV=api, NEXT_PUBLIC_API_URL=/api/v1 and API_INTERNAL_URL=http://127.0.0.1:8000.
Backend: FRONTEND_URL=http://localhost:3000; leave DATABASE_URL empty for SQLite; STORAGE_PROVIDER=local;
EMAIL_PROVIDER=outbox; AI_PROVIDER=local; SPEECH_PROVIDER=disabled.

Completion check

The website loads, /api/v1/health responds through the frontend and a saved record survives a backend restart.

Step 6 - Design the records and database

Current database: server/db.ts creates records, locks and migrations. Most domain objects are JSON text in records, keyed by (kind,id), with campus_id and owner_id for scoping. The logical entities below are not separate SQL tables in the current implementation.

LOGICAL ENTITY	IMPORTANT FIELDS / RELATIONSHIPS
Account + session	Account ID, email, password hash, role, campus, approval; session hash, user ID, CSRF and expiry.
Campus + organization	Institution identity and registered campus ID; recruiter organization identity and approvals.
Student profile	User/campus IDs, course, branch, graduation year, CGPA, backlogs, skills, projects and evidence.
Assessment + attempt	Question/content IDs, actor ID, responses, graded score, completion time and award state.
Drive + application	Recruiter owner, allowed campuses, criteria and state; student/drive link and selection progress.
Schedule + round	Date/time, room, recruiter, panel, student, status and round outcome.
Offer + document	Student/application links, offer type, response, joining progress; private storage path and verification.
Notification + job + audit	Recipient, event, read state; delivery attempts and deduplication; actor and changed fields.

Build step

Create a small database adapter with get, list, put, remove and transaction methods. Use parameterized SQL. Every scoped record needs the correct campus and owner IDs. Handle rollback so a failed operation leaves no partial application, offer or notification.

Recommended production evolution

As volume grows, migrate selected entities to normalized PostgreSQL tables with foreign keys and unique constraints, such as one application per student/drive. Use a staged migration and validate counts and access before removing old records. Keep transaction boundaries when changing storage design.

Current PostgreSQL transactions use a shared placement lock; it preserves consistency but can serialize unrelated work. Benchmark before replacing it with finer resource locks. The code enables row-level security, but application scope checks still matter; validate the database role and policies used by a deployment.

Completion check

Cross-campus reads fail; concurrent writes remain consistent; migrations can run repeatedly and backups can be restored.

Step 7 - Build authentication and access

Implement in this order

- **Registration:** validate email, password, role and registered campus membership. Normalize account lookup. Put new campus/recruiter accounts into the appropriate approval flow.
- **Password storage:** use salted scrypt hashing through Node crypto, as in server/auth.ts. Never store or return plaintext passwords.
- **Login:** verify the password, generate a random session token, store its hash and expiry, and set an HTTP-only cookie. Current sessions last one day, or 30 days with remember-me.
- **Current account:** GET /auth/me returns server identity and the session CSRF token. Protected writes send X-CSRF-Token.
- **Authorization:** check authenticated role, approval, campus membership, record ownership and allowed state for every operation.
- **Recovery/logout:** revoke sessions on logout; issue expiring single-use reset tokens; rate-limit authentication requests.

Important API contracts

```
POST /api/v1/auth/register
POST /api/v1/auth/login
GET  /api/v1/auth/me
POST /api/v1/auth/logout
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
```

Use strict origin validation and secure cookies in production. The current system uses cookie sessions, not browser-stored JWTs. Do not accept a client-provided role or student ID as proof of access. X-Student-ID is only a selector after the server verifies the staff member can access that student.

Account administration in the existing project

```
npm run server:build
npm run db:approve -- campus-account@example.edu
npm run admin:grant -- admin-account@example.edu
# Remove the permission when needed:
npm run admin:revoke -- admin-account@example.edu
```

The addresses above are examples. Complete the actual account profile first, then sign out and sign back in after granting administrator access. Google sign-in is a later optional integration; the core email/password flow should work first.

Completion check

An unapproved recruiter cannot run protected placement actions; a student cannot access a staff operation or another student's private record.

Step 8 - Student profiles and private uploads

Profile flow

Build a profile form for name, course, branch, graduation year, CGPA, active backlogs, bio, skills, projects and certifications. Separate student-entered information from verified evidence. Validate numeric ranges and mandatory fields both in the form and on the server.

The student sees their own profile. Campus editors can update allowed academic/profile fields for students in the same campus. Their edits must preserve verification, points, identity and existing unrelated records, and create an audit entry.

Upload flow

- The student sends multipart form data to POST /api/v1/documents with file and type. The current limit is 10 MB.
- Check authorization, size, supported format and file signature. Supported formats in the API reference are PDF, PNG, JPEG and WebP.
- Store the bytes in local private storage for development or a private Supabase bucket in production. Save only the file reference and metadata with the student record.
- Serve GET /documents/:id/download only after checking access. Never place resumes or certificates in public/.
- Use pdf-parse for supported PDF text extraction. Handle image-only, damaged or unreadable files gracefully; OCR would be a separate enhancement.
- Verify documents through an authorized campus action. When replacing a file, queue cleanup after the record transaction commits.

Technology-to-file mapping

CONCERN	LOCATION / TOOL
Profile screen and campus editor	src/features/profile.tsx and institute-student-editor.tsx
Shared profile contracts	src/types and src/utis/student-records.ts
Upload route and storage provider	server/app.ts, server/storage.ts; Multer and private storage
Server operations and audit	server/workspace.ts and server/services.ts

Completion check

The student can save and reload a profile, authorized staff can verify evidence, and an unrelated account cannot download private files.

Step 9 - Assessments and readiness scoring

Build assessments before trusting a readiness score

Create question banks, assessment definitions, attempts and graded results. Administrators manage content. The server sends questions without answer keys and grades submitted options. Record each successful completion once so replaying a request cannot award extra points.

Build assessment and communication screens with clear progress, submitted state, validation and retry behavior. Keep practice feedback separate from verified academic or hiring evidence.

READINESS CATEGORY	WEIGHT	EVIDENCE
Verified skills	30%	Skills with recorded verification
Academics	20%	Recorded academic profile
Projects	15%	Project evidence
Aptitude	15%	Recorded aptitude results
Communication	10%	Recorded communication evidence
Interviews	10%	Recorded interview evidence

Calculation and explanation

Each normalized category is on a 0-100 scale. Overall readiness is the weighted sum; missing evidence contributes zero. For example, category scores of 80, 70, 70, 75, 50 and 65 yield 72.25, rounded to **72**: $24 + 14 + 10.5 + 11.25 + 5 + 6.5$.

Levels: below 45 = Not Ready; 45-69 = Developing; 70-84 = Ready; 85-100 = Highly Employable. The current code rounds the weighted total to an integer before assigning its level.

Normalization: skills = rounded verified/total skill percentage; academics = CGPA * 10 minus 10 per active backlog, bounded to 0-100; projects = 35 per project, capped at 100. Aptitude, communication/soft and interview use rounded averages of the matching recorded attempt scores.

Show category contributions and distinguish a zero-scored attempt from missing evidence. Explain how to improve specific gaps. Readiness is an evidence summary, not a validated probability of getting a job.

Current implementation map

Scoring: src/utls/scoring.ts. Assessment UI: src/features/assessments.tsx. Questions: src/mocks/question-banks.ts and server-managed content. Contest achievements: src/utls/contest-achievements.ts. Successful contest completions and IST calendar days drive streaks; multiple completions on one day count as one streak day.

Completion check

Answer keys remain private, repeat submission does not duplicate awards, missing evidence is visible and the score can be independently recomputed.

Step 10 - Eligibility and applicant matching

First enforce hard eligibility

A student must pass campus, course, branch, graduation year, minimum CGPA, maximum active backlogs and optional mandatory-skill conditions. Additional free-text requirements need recorded campus verification. Save and display clear failure reasons. A high ranking score must never override an eligibility failure.

FIT COMPONENT	WEIGHT	MEANING
Required skills	35%	Alignment with required job skills
Preferred skills	10%	Additional desired skills
Assessments	20%	Average of recorded assessment scores
Relevant projects	15%	Project alignment with the role
Certifications	5%	50 points per recorded certification, capped at 100
Academics	15%	Academic suitability

Current matching gives 0.5 credit to recorded unverified required skills and 1.0 credit to verified skills. With no preferred skills, that component follows required-skill alignment. Relevant projects contribute 50 points each, capped at 100; academics use $\text{CGPA} * 10$, capped at 100. The weighted fit total is rounded. See `src/utls/scoring.ts` for normalization.

Then rank eligible applicants

Final ranking = $0.85 * \text{fit score} + 0.15 * \text{NLP relevance}$

Example: fit 80, normalized relevance 60

Final ranking = $68 + 9 = 77$

Local NLP uses TF-IDF cosine relevance and a reviewed skill dictionary. It extracts suggested skills and selected academic conditions from job descriptions. Let the recruiter/campus review extracted requirements; job text is not a verified student fact.

Implementation and evaluation

Use `src/utls/placement.ts` for eligibility, `src/utls/scoring.ts` for fit, `src/utls/skills.ts` and `server/nlp.ts`. Expose authorized `GET /api/v1/drives/:id/matches` with eligibility reasons, factors and gaps. The server rounds the hybrid rank. Use stable tie handling and explain the ordering.

The existing demo covers 13 synthetic profiles, three drives, 39 manually labelled eligibility pairs and three expected top-ranked candidates. It is a small workflow benchmark, not real hiring accuracy. Run `npm run demo:placement` and inspect `output/placement-demo-report.json`.

Completion check

Ineligible students remain excluded, rankings are reproducible and users can see the evidence behind each recommendation.

Step 11 - Drives, scheduling and rounds

Drive state machine

DRAFT -> SUBMITTED -> UNDER_REVIEW -> SCHEDULING
-> AWAITING_RECRUITER_CONFIRMATION -> CONFIRMED
-> campus finalization -> ACTIVE
-> IN_PROGRESS -> COMPLETED

Treat campus finalization as a guarded action, not an extra stored state. Add permitted review changes, rejection, cancellation and resubmission paths. Validate the actor and current state on the server before every transition.

Create the drive

Collect recruiter organization, job title, description, campus scope, academic criteria, required/preferred skills, rounds and proposed timing. Persist draft data, submit for review, then expose only active opportunities to students. Check duplicate applications and store applications against both student and drive IDs.

Make scheduling conflict-aware

- Represent times as unambiguous timestamps and display them in the intended local timezone. Define whether touching intervals count as overlapping.
- Check rooms/resources, recruiter availability and overlapping cohorts at drive level. Check candidate, room and panel availability at interview level.
- Campus requests suggestions through POST /drives/:id/schedule-suggestions. Current behavior returns three suggestions for a valid schedule request.
- Recheck every reservation in the save transaction. Suggestions do not reserve a slot; another request may take it before saving.
- Require recruiter confirmation and campus finalization before activation. Record cancellations and changes with audits and targeted notifications.

Selection rounds

Record the application's current round, round outcomes and authorized progression. Recruiters must only operate on their own drives/applicants. Do not permit arbitrary client state patches or an Offer status before required progression.

Current files

Drive UI/domain: src/features/drives.tsx and src/services/drive.*. Interviews: src/features/interviews.tsx. Trusted workflows: server/workspace.ts and server/recruitment.ts. Scope policies: server/services.ts.

Completion check

Two simultaneous attempts cannot reserve the same conflicting resource; an invalid state transition or unrelated recruiter request fails.

Step 12 - Notifications, offers and joining

Reliable notifications

Save an in-app notification and its email job together with the workflow change. Email delivery runs after commit, with retry and deduplication. If the email provider is unavailable, the placement record must remain valid and the job can be retried.

Development uses the outbox and sends no external email. In the existing project, `npm run email:preview` shows local messages after the backend CLI is built. Password-reset links in `server/data/outbox.json` are private.

Offer lifecycle

- Recruiter releases an offer only after the application reaches the permitted Offer stage. Store type, student/application link, compensation details and response/joining information.
- Support Full-time, PPO and Internship conversion. Students can accept, decline, defer or withdraw through valid actions.
- Keep offer release, acceptance and actual joining as separate milestones. Campus verifies the required documents and records joining.
- Current joining requires a verified resume and no outstanding unverified documents. Recheck these conditions inside the joining operation.

Background work

WORKER	RESPONSIBILITY	BUILD RULE
Email delivery	Send queued provider messages	Retry transient failures; store attempts; prevent duplicate sends.
Reminders	Interview, joining and document follow-up	Deduplicate daily and target only relevant recipients.
File cleanup	Remove superseded private objects	Queue after commit; retry without corrupting current document references.

Current code: `server/mail.ts`, `server/reminders.ts`, `server/storage.ts` and `server/recruitment.ts`. A persistent backend process is required for timely reminders and email work. Sleeping hosting services pause those workers.

Recommended before multiple worker replicas

Use a durable shared queue or atomically claimed database jobs, with visibility timeouts and idempotent handlers. Do not run several uncoordinated copies of a periodic worker against the same job list.

Completion check

A provider failure does not lose the offer; retries do not duplicate notifications; joining is blocked until document conditions are satisfied.

Step 13 - Analytics and optional intelligence

Build reliable analytics from stored outcomes

Create GET /api/v1/analytics with campus-wide or recruiter-owned results. Track eligible students, applicants, selection progress, offers released, offers accepted, joining and document status. Use a clear denominator and period for each metric.

METRIC	DEFINITION TO DISPLAY
Offer acceptance rate	Accepted offers / released offers within the defined period.
Joining conversion	Recorded joinings / accepted offers in the selected cohort.
Student placement rate	Distinct placed students / defined eligible student cohort; explain what counts as placed.
Readiness distribution	Students by evidence-based readiness level; show missing evidence.
CTC / compensation	Specify offer type, currency, annual basis, missing values and whether duplicate offers are included.

Compute distinct-student measures separately from counts of offers. Exclude cancelled/invalid records according to written rules. Show empty states and last-updated time. The current dashboards refresh relevant placement analytics approximately every 15 seconds; this is polling, not a streaming architecture.

Optional coaching and voice

Current provider interfaces include local preparation analysis, optional external generative coaching and optional transcription. Core typed preparation should keep working when an external provider is disabled or fails. Require explicit student consent before external analysis or voice processing. Do not claim pronunciation or accent assessment from basic transcript checks.

Optional outcome prediction

The existing model path uses regularized logistic regression over six readiness features. Training requires at least 60 labelled historical outcomes across three cohorts and holds out the latest cohort. Report accuracy, precision, recall and Brier score. Both training and test data need positive and negative outcomes.

This minimum is an implementation gate, not proof that a model is valid for deployment. Evaluate calibration, leakage, drift and campus differences on real data. The app rejects synthetic-trained live prediction models. Real outcome data is still needed before claiming placement-prediction performance.

Completion check

Analytics can be traced to records; optional intelligence is consent-based and never overrides authorization or eligibility.

Step 14 - Connect the frontend to the API

Build each feature as a vertical slice

- Define the response/input contract in `src/types`. Keep stable field names and distinguish missing data from zero values.
- Create the backend operation with schema validation, permissions and transaction handling. Add it to the explicit service allowlist.
- Expose a typed client call in `src/services`. The current service adapter uses `POST /services/:service/:method` with a JSON body containing args.
- Create a query hook for reading and a mutation for saving. After success, invalidate the profile, application, drive or analytics queries affected by the change.
- Render loading, empty, validation, permission, network-error and success states. Disable accidental duplicate submissions while a write is pending.

Request contracts used in the current app

Base URL: `/api/v1`
Cookie: sent with requests
Write header: `X-CSRF-Token: <session token>`
Service body: `{ "args": [<validated arguments>] }`
Optional staff selector: `X-Student-ID: <authorized student>`

Do not invent unsupported routes for business operations. Use `server/API.md` and `server/services.ts` to confirm the service method and role policy. Public GET endpoints do not need a fabricated write token; authenticated writes follow the session CSRF rules.

Role dashboards

Student: readiness, gaps, opportunities and personal offers. Recruiter: matches, confirmation, rounds and offers. Campus: readiness support, approvals, resources and outcomes. Administrator: accounts, campuses, content and audits. Shared cards are reusable, but each role needs its own actions and layout.

Usability checklist

Make controls keyboard-accessible, label inputs and errors, show focus, keep table views readable on mobile, and provide text labels for critical icon actions. Use reduced-motion preferences. Mask private file paths and avoid showing provider secrets in integration screens.

Completion check

A user can finish the main journey using the screens, receive useful failures and see saved changes after refreshing or signing in again.

Step 15 - Test the real risks

The current repository has build/type/lint checks and a placement demo, but its automated test files and runner have been removed. The scenarios below are a recommended test plan, not a claim that a test suite already exists.

TEST AREA	IMPORTANT SCENARIO	EXPECTED RESULT
Authentication	Wrong password; expired session; forged role	Access denied with safe error; no privilege change.
Campus isolation	Campus A requests Campus B student or document	Read/write denied, including with X-Student-ID.
Eligibility	CGPA at boundary; backlog exceeded; missing skill	Correct eligible/ineligible result and explicit reasons.
Assessment integrity	Inspect payload; replay completion	No answer key exposed; no duplicate points.
Schedule contention	Two overlapping saves sent simultaneously	One valid reservation; conflicting request rejected.
Workflow integrity	Activate draft; release offer too early	Invalid transitions rejected with no partial writes.
Uploads / joining	Wrong signature; oversized file; unverified resume	Upload rejected or joining blocked as appropriate.
Worker reliability	Provider timeout; replay same job	Record stays valid; controlled retry without duplicates.
Analytics	Student has several offers; missing compensation	Correct distinct counts and explicitly defined handling.

Existing project checks

```
npm run typecheck
npm run server:typecheck
npm run lint
npm run server:build
npm run build
npm run demo:placement
```

Run each check and inspect its result. Then test the built frontend and backend together using actual accounts in a development/staging database. The synthetic demo uses an isolated in-memory database and simulated document records; it does not prove real upload, email or production load behavior.

Recommended tooling

Use an HTTP client for contract exploration, automated API integration tests for permission/transaction cases, and browser tests for one full user journey. Choose compatible test tools when adding a runner; prioritize these risks over duplicating implementation details.

Completion check

All checks pass, core access and concurrency cases are tested, and remaining known limitations are documented before release.

Step 16 - Deploy the application

1. Prepare production services

Create PostgreSQL, a private Supabase Storage bucket, and a Resend configuration with a verified sender. Confirm the database CA matches the provider and require certificate/hostname validation. Keep real secrets only in backend hosting settings. Preserve a separate staging environment.

2. Deploy the Express API to Render

Root directory: repository root
 Build: `npm ci --include=dev && npm run server:build`
 Start: `npm run server:start`
 Health check: `/api/v1/health`
 Host: `0.0.0.0`
 Port: use the platform-provided PORT

Use `render.yaml` as the project reference and official Render instructions [4]. Set `NODE_ENV=production`, `FRONTEND_URL` to the exact frontend HTTPS origin, `DATABASE_URL`, optional `DATABASE_CA_PATH`, `STORAGE_PROVIDER=supabase`, `SUPABASE_URL`, `SUPABASE_SECRET_KEY`, `SUPABASE_STORAGE_BUCKET`, `EMAIL_PROVIDER=resend`, `RESEND_API_KEY` and `EMAIL_FROM`.

3. Deploy the frontend to Vercel

`NEXT_PUBLIC_APP_ENV=api`
`NEXT_PUBLIC_API_URL=/api/v1`
`API_INTERNAL_URL=https://<actual-backend-host>`

Use the backend origin without `/api/v1` and without a trailing slash. The frontend rewrites API requests to this host so the browser uses its frontend domain. Redeploy after changing `API_INTERNAL_URL` because the rewrite is configured during the build. Update backend `FRONTEND_URL` when the frontend origin changes.

4. Validate and release

Test health through the frontend, registration/login/logout, approvals, password recovery, upload/download authorization, a complete drive journey and real email delivery in staging. Confirm production records survive restart and a backup can be restored. Grant administrator access through the operator CLI to the intended account.

Current tables are initialized at backend startup. For future structural migrations, plan compatibility and rollback before deploying. The included Render configuration selects a free plan; choose hosting availability that meets actual worker and response-time needs. This guide does not estimate current pricing.

Completion check

The deployed frontend talks to the correct backend, secure sessions work, storage is private and workers deliver expected events.

Step 17 - Maintain, scale and hand over

Operate the first release

- Monitor API health, error rate, slow database queries, email backlog, worker failures and storage growth. Avoid logging passwords, reset links or session tokens.
- Schedule database backups and private-file recovery checks. Define how long records and documents are retained, and who can request correction or deletion.
- Maintain a dependency-update process with staging verification. Keep environment templates up to date without putting real secrets into them.
- Document how to approve staff, grant/revoke admin access, rotate credentials, restore data and roll back a release.

Scale only after measuring

Start with one authoritative API and PostgreSQL database. Index actual campus/owner query patterns and paginate large lists. Measure simultaneous applications, schedule reservations, database latency and notification delays using realistic cohorts.

Before adding API replicas, use a shared rate-limit store and coordinated durable workers. Private file storage can scale separately. Review the current global placement transaction lock when it becomes a measured bottleneck. A small demo does not establish a concurrent-user capacity.

Suggested 8-week schedule

WEEK	DELIVERABLE
1	Requirements, sketches, contracts, scaffolding and health route.
2	Database, sessions, approvals and role/campus isolation.
3	Student profiles, evidence, private uploads and verification.
4	Assessments, scoring, skill gaps and explainable matching.
5	Drives, applications, scheduling and guarded round progression.
6	Offers, joining, notifications, reminders and analytics.
7	Risk-focused checks, staging deployment and usability fixes.
8	Production release, recovery exercise, report and handover.

This is a planning estimate for a developer already comfortable with the stack, not a guaranteed delivery date. Beginners should allocate learning time; integrations and concurrency bugs can take longer.

Completion check

Another developer or operator can install, configure, run, verify, deploy and recover the system using the handover material.

Final checklist and source references

What a complete handover includes

- Requirements and role matrix; architecture and logical data model; explicit workflow/state rules.
- Source code and dependency lockfile; sample configuration; local run and deployment instructions.
- API contracts; approval/admin instructions; private-storage and email setup; backup/restore procedure.
- Validation results and reproducible demo data; known limitations; model provenance and evaluation if prediction is enabled.
- A recorded walkthrough: profile -> matching -> scheduling -> notification -> offer tracking -> analytics.

Repository evidence reviewed

Paths below are relative to C:/CampusLink. This is an implementation snapshot, not an independent security or production-readiness audit.

SOURCE	WHAT IT SUPPORTS
README.md and server/API.md	Setup, role responsibilities, endpoint contracts, algorithms and deployment flow.
package.json	Declared technologies, runtime requirement and available scripts.
server/db.ts and server/auth.ts	Generic scoped records, transaction locking, script and cookie-session records.
.env.example / server/.env.example	Local provider defaults and configuration variable names.
next.config.mjs / render.yaml / server/build.cjs	API rewrites, backend build/start configuration and hosting settings.
server/ml.ts	Outcome features, cohort holdout and historical training gates.

Official technology references

Checked 8 October 2026. These links support the referenced technology/setup notes; the detailed project rules above come from the repository.

[1] [Next.js installation and project setup](https://nextjs.org/docs/app/getting-started/installation)

<https://nextjs.org/docs/app/getting-started/installation>

[2] [Express installation](https://expressjs.com/en/starter/installing/)

<https://expressjs.com/en/starter/installing/>

[3] [Supabase public and private storage buckets](https://supabase.com/docs/guides/storage/buckets/fundamentals)

<https://supabase.com/docs/guides/storage/buckets/fundamentals>

[4] [Deploy an Express application on Render](https://render.com/docs/deploy-node-express-app)

<https://render.com/docs/deploy-node-express-app>